Node.js — Day 28

Small Node.js Practice Project — User Management CLI

■ What We Built

Day 28 was an integration day: we combined Node.js concepts from earlier lessons into one small command-line project. The goal was to understand the architecture and be able to rebuild it step by step later.

The application accepts a name and age, validates the input, creates a user, determines a status, and prints the result.

```
node app.js Bhavana 26

Name: Bhavana
Age: 26
Status: Adult
```

■ Project Structure

```
day28/
■■■ node-user-app/
    ■■■ app.js
    ■■■ validation.js
    ■■■ userService.js
    ■■■ user.js
```

Memory rule: app.js coordinates → validation.js validates → userService.js handles the user operation → user.js creates the user and applies business logic.

1. Step 1 — Create the Project

Create a folder named **node-user-app**. Inside it create four files: app.js, validation.js, userService.js, and user.js. We deliberately keep responsibilities separate instead of putting everything into app.js.

2. Step 2 — Read Command-Line Input

Use process.argv to receive the name and age. Command-line arguments arrive as strings, so convert the age with Number().

```
const name = process.argv[2];
const age = Number(process.argv[3]);

console.log("Name:", name);
console.log("Age:", age);
```

For **node app.js Bhavana 26**, process.argv[2] is Bhavana and process.argv[3] is "26". Number("26") produces the number 26.

3. Step 3 — Put Validation in validation.js

```
function validateUser(name, age) {
    if (!name) {
        return false;
    }

    if (Number.isNaN(age) || age < 0) {
        return false;
    }

    return true;
}

module.exports = validateUser;
```

The function returns true for input that passes the rules and false when the input is invalid. module.exports makes the function available to another file.

> Why separate this? app.js should coordinate the validation rather than contain every validation rule. This keeps the code easier to read, organize, maintain, and extend.

4. Step 4 — Connect Validation to app.js

```
const validateUser = require("./validation");

const name = process.argv[2];
const age = Number(process.argv[3]);

if (!validateUser(name, age)) {
    console.log("Please provide name and age");
    process.exit();
}
```

app.js asks the validation module whether the input is valid. If validation returns false, the program prints the message and stops. app.js does not need to know the internal validation rules.

5. Step 5 — Add Business Logic in user.js

user.js is responsible for creating the user and determining the user's status.

```
function createUser(name, age) {
    let status;

    if (age < 18) {
        status = "Minor";
    } else if (age <= 59) {
        status = "Adult";
    } else {
        status = "Senior";
    }

    return {
        name: name,
        age: age,
        status: status
    };
}

module.exports = createUser;
```

Business rules: age below 18 → Minor; age 18–59 → Adult; age 60 or above → Senior.

> We use **else if** so only one status branch is selected. With separate if statements, a younger user's status could be overwritten by a later condition.

6. Step 6 — Add userService.js

The service layer provides a clean place for the user-related operation. It loads createUser from user.js and exposes registerUser to app.js.

```
const createUser = require("./user");

function registerUser(name, age) {
    const user = createUser(name, age);

    return user;
}

module.exports = registerUser;
```

The **./** in require("./user") tells Node.js that user.js is a local module in the current folder.

7. Step 7 — Final app.js

```
const validateUser = require("./validation");
const registerUser = require("./userService");

const name = process.argv[2];
const age = Number(process.argv[3]);

if (!validateUser(name, age)) {
    console.log("Please provide name and age");
    process.exit();
}

const user = registerUser(name, age);

console.log("Name:", user.name);
console.log("Age:", user.age);
console.log("Status:", user.status);
```

> This is the final coordinator. It receives input, validates it, calls the service, receives the user object, and prints selected fields.

8. Complete Flow

```
Terminal
   ↓
process.argv
   ↓
app.js
   ↓
validation.js
   ↓
valid? ■■ No → error + stop
   ■
  Yes
   ↓
userService.js
```

```
   ↓
user.js
   ↓
User object
   ↓
app.js
   ↓
Output
```

This is the mental model to remember. Each module hides its internal details behind a simple interface. That is the abstraction idea you identified during the final assessment.

9. Module Responsibilities

File	Main responsibility
app.js	Coordinates input, validation, service calls, and output.
validation.js	Checks whether the input is valid.
userService.js	Handles the user-related operation and connects app.js to user logic.
user.js	Creates the user and contains the age/status business logic.

10. Testing — Normal Inputs

```
node app.js Bhavana 16
→ Name: Bhavana
→ Age: 16
→ Status: Minor

node app.js Bhavana 26
→ Name: Bhavana
→ Age: 26
→ Status: Adult

node app.js Ravi 65
→ Name: Ravi
→ Age: 65
→ Status: Senior
```

Testing multiple age ranges confirmed that the business logic correctly selects the expected status.

11. Testing — Invalid Inputs

```
node app.js
→ Please provide name and age

node app.js Bhavana abc
→ Please provide name and age

node app.js Bhavana -5
→ Please provide name and age
```

These tests show why validation is necessary. Missing input is rejected, "abc" becomes NaN when converted with Number(), and a negative age is rejected even though it is technically a number.

12. Rebuild the Project Years Later — Exact Sequence

If you return to these notes after 10 years, do not try to memorize the entire code. Follow this build order:

```
1. Create node-user-app/

2. Create:
   app.js
   validation.js
   userService.js
   user.js

3. app.js:
   Read process.argv[2] as name.
   Read process.argv[3] and convert with Number().

4. validation.js:
   Check that name exists.
   Check Number.isNaN(age).
   Check age < 0.
   Return true/false.
   Export validateUser.

5. user.js:
   Create createUser(name, age).
   Decide Minor / Adult / Senior.
   Return { name, age, status }.
   Export createUser.

6. userService.js:
   require("./user").
   Create registerUser(name, age).
   Call createUser().
   Return the user.
   Export registerUser.

7. app.js:
   require validation.js.
   require userService.js.
   Validate.
   Stop on invalid input.
   Call registerUser().
   Print user.name, user.age, user.status.

8. Test:
   16 → Minor
   26 → Adult
   65 → Senior
   abc → rejected
   -5 → rejected
```

> This sequence is more important than memorizing individual lines. It tells you **what to build first, where each responsibility belongs, and how the modules connect.**

13. Key Concepts to Carry Forward

• **process.argv** — receives command-line arguments.

• **Number()** — converts the age argument into a number.

• **require()** — loads another module.

• **module.exports** — exposes a function from a module.

• **Validation** — rejects bad input before processing.

• **Business logic** — contains application rules and decisions.

• **Service layer** — organizes operations around a domain such as users.

• **Separation of responsibilities** — each file has a focused job.

• **Abstraction** — app.js can use a module without knowing its internal implementation.

14. Day 28 Final Assessment

You answered the final assessment in your own words and demonstrated understanding of the project architecture.

Q1 — Complete flow: You correctly explained process.argv → app.js → validation.js → userService.js → user.js → result → app.js output. You also identified app.js as the coordinator and connected this to abstraction.

Q2 — Responsibilities: You correctly identified app.js as the coordinator, validation.js as the validation module, userService.js as the user-operation layer, and user.js as the user/business-logic module.

Q3 — Validation: You correctly explained that Number.isNaN(age) detects NaN and age < 0 rejects negative ages.

■ Assessment: 3/3 — PASSED

■ DAY 28 — COMPLETE

You built the project incrementally, tested normal and invalid inputs, corrected a business-logic issue, added explicit validation for NaN and negative ages, and explained the architecture yourself.

> Most importantly, you now have a repeatable method for building a small Node.js application instead of relying on one large file.

■ Next: Final Node.js Revision + Assessment

The next session will review the Node.js foundation and assess the concepts you have learned before moving forward to Express.js.